

# CHAPTER 03

## OpenCV 주요 클래스

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

# 3장 OpenCV 주요 클래스

---

3.1 기본 자료형 클래스

3.2 Mat 클래스

3.3 Vec과 Scalar 클래스

3.4 InputArray와 OutputArray 클래스

## 3.2 Mat 클래스

---

# Mat 클래스 개요

- OpenCV 라이브러리에서 가장 많이 사용하는 클래스
- 일반적인 2차원 행렬뿐만 아니라 고차원 행렬을 표현할 수 있음
- 2차원 영상 데이터를 저장하고 처리하는 용도로 가장 많이 사용되고 있음, 그레이스케일 또는 컬러 영상을 저장할 수 있음
- Mat 클래스는 다양한 형태의 생성자와 많은 멤버 함수, 멤버 변수를 가지고 있음
- Mat 클래스의 모든 멤버 변수는 public 접근 지정자로 선언되어 있어서 클래스 외부에서 직접 접근할 수 있음

# Mat 클래스 개요

## 코드 3-6 간략화한 Mat 클래스 정의

```
01  class Mat
02  {
03  public:
04      Mat();
05      Mat(int rows, int cols, int type);
06      Mat(Size size, int type);
07      Mat(int rows, int cols, int type, const Scalar& s);
08      Mat(Size size, int type, const Scalar& s);
09      Mat(const Mat& m);
10      ~Mat();
11
12      void create(int rows, int cols, int type);
13      bool empty() const;
14
15      Mat clone() const;
16      void copyTo(OutputArray m) const;
17      Mat& setTo(InputArray value, InputArray mask=noArray());
```

# Mat 클래스 개요

## 코드 3-6 간략화한 Mat 클래스 정의

```

19     static MatExpr zeros(int rows, int cols, int type);
20     static MatExpr ones(int rows, int cols, int type);
21
22     Mat& operator = (const Mat& m);
23     Mat operator()( const Rect& roi ) const;
24
25     template<typename _Tp> _Tp* ptr(int i0 = 0);
26     template<typename _Tp> _Tp& at(int row, int col);
27
28     int dims;
29     int rows, cols;
30     uchar* data;
31     MatSize size;
32     ...
33 };

```

# Mat 클래스 개요

- `Mat::dims` : Mat 행렬의 차원을 나타냄(`dims ≥ 2`), 영상과 같은 2차원 행렬의 경우 2임
- `Mat::rows` : 행렬의 행의 개수, 영상의 경우 영상의 세로 픽셀 크기임
- `Mat::cols` : 행렬의 열의 개수, 영상의 경우 영상의 가로 픽셀 크기임
- `Mat::rows`, `Mat::cols`는 2차원 행렬인 경우에만 의미 있는 값을 가짐, 3차원 이상의 행렬에서는 -1이 저장됨
- `Mat::size` : 3차원 이상 행렬의 크기 정보를 저장
- `Mat::data` : 행렬의 원소 데이터가 저장되어 있는 메모리 공간의 주소를 저장, 아무것도 저장되어 있지 않다면 0(NULL) 값을 가짐
- Mat 클래스의 멤버변수에는 행렬의 특성정보만 저장하고 행렬의 원소 데이터는 동적 할당된 별도의 메모리에 저장한다.

# Mat 클래스 개요

- 행렬 : 숫자들을 직사각형(2차원) 형태로 배열해 놓은 것
- 행렬의 원소 : 행렬의 구성 요소, 행과 열의 수로 위치표현
- 4행 5열의 행렬의 예

|    | 1열 | 2열 | 3열 | 4열  | 5열 |
|----|----|----|----|-----|----|
| 1행 | 0  | 10 | -5 | 4   | 2  |
| 2행 | 2  | 1  | 2  | 5   | 4  |
| 3행 | 5  | -7 | 0  | -12 | -6 |
| 4행 | 22 | 5  | 3  | 9   | 8  |

4행 1열의 원소 : 22  
3행 3열의 원소 : 0

Mat 객체(특성정보)

```
dims =>2
rows =>4
cols =>5
data =>xxx
...
```

정적할당

행렬원소데이터

```
0,10,-5,4,2,
2,1,2,5,4,
5,-7,0,-12,-6,
22,5,3,9,8
```

동적할당



# Mat 클래스 개요

- 행렬 원소의 깊이(depth) : 행렬 원소의 자료형
- 행렬은 다양한 자료형의 원소를 가질 수 있음, 기본 자료형 중에서 unsigned char, signed char, unsigned short, signed short, int, float, double 자료형을 가질 수 있음
- 행렬 원소의 깊이(depth)는 다음과 같은 기호상수(매크로 상수)를 이용하여 정의함

```
#define CV_8U    0    // uchar, unsigned char
#define CV_8S    1    // schar, signed char
#define CV_16U   2    // ushort, unsigned short
#define CV_16S   3    // signed short
#define CV_32S   4    // int
#define CV_32F   5    // float
#define CV_64F   6    // double
#define CV_16F   7    // float16_t
```

# Mat 클래스 개요

- 깊이(depth)를 정의하는 기호상수(매크로상수)의 형식

`CV_<bit-depth>{U|S|F}`

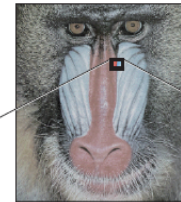
- CV\_ : OpenCV를 나타내는 접두사와 같은 역할임
- <bit-depth> : 8, 16, 32, 64의 숫자 중 지정, 원소 값의 비트 수를 나타냄
- {U|S|F} : U, S, F 세 문자 중 하나를 지정, 여기서 U는 unsigned형, S는 signed형, F는 부동 소수형(float, double)을 의미함

# Mat 클래스 개요

- 행렬 원소는 하나의 값 또는 여러 개의 값을 가질 수 있음
- 그레이스케일 영상은 픽셀이 밝기값 1개의 값을 가짐 -> 2차원 행렬로 표현할 때 행렬의 원소는 1개의 값을 가짐
- 컬러영상은 픽셀이 B, G, R값 3개의 값을 가짐 -> 2차원 행렬로 표현할 때 행렬의 원소는 3개의 값을 가짐



|     |     |     |     |     |     |    |    |    |    |
|-----|-----|-----|-----|-----|-----|----|----|----|----|
| 187 | 187 | 187 | 194 | 197 | 173 | 77 | 25 | 19 | 19 |
| 190 | 187 | 190 | 191 | 158 | 37  | 15 | 14 | 20 | 20 |
| 187 | 182 | 180 | 127 | 32  | 16  | 13 | 16 | 14 | 12 |
| 184 | 186 | 172 | 100 | 20  | 13  | 15 | 18 | 13 | 18 |
| 186 | 190 | 187 | 127 | 18  | 14  | 15 | 14 | 12 | 10 |
| 189 | 192 | 192 | 148 | 16  | 15  | 11 | 10 | 10 | 9  |
| 192 | 195 | 181 | 37  | 13  | 10  | 10 | 10 | 10 | 10 |
| 189 | 194 | 54  | 14  | 11  | 10  | 10 | 10 | 9  | 8  |
| 189 | 194 | 19  | 16  | 11  | 11  | 10 | 10 | 9  | 9  |
| 192 | 88  | 12  | 11  | 11  | 10  | 10 | 10 | 9  | 9  |



|     |     |     |     |     |     |     |     |     |     |     |    |     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 231 | 103 | 92  | 228 | 118 | 86  | 220 | 122 | 122 | 213 | 92  | 59 | 166 | 136 | 164 | 131 | 185 | 228 | 142 | 192 | 227 | 145 | 195 | 228 |
| 229 | 104 | 92  | 221 | 114 | 72  | 226 | 109 | 109 | 208 | 101 | 51 | 150 | 146 | 189 | 137 | 189 | 232 | 132 | 189 | 229 | 143 | 195 | 229 |
| 228 | 109 | 103 | 229 | 110 | 90  | 228 | 107 | 107 | 206 | 86  | 64 | 144 | 165 | 206 | 138 | 187 | 230 | 137 | 188 | 227 | 142 | 195 | 230 |
| 229 | 110 | 104 | 231 | 108 | 102 | 226 | 117 | 102 | 206 | 81  | 45 | 134 | 155 | 199 | 135 | 187 | 230 | 139 | 193 | 231 | 156 | 199 | 229 |
| 224 | 106 | 93  | 219 | 132 | 114 | 208 | 118 | 137 | 189 | 88  | 70 | 133 | 166 | 211 | 127 | 184 | 229 | 145 | 192 | 229 | 165 | 201 | 228 |
| 231 | 111 | 98  | 227 | 102 | 70  | 209 | 110 | 103 | 178 | 84  | 65 | 121 | 157 | 210 | 131 | 183 | 229 | 145 | 192 | 228 | 161 | 199 | 228 |

$$gray \Rightarrow \begin{pmatrix} 187 & \dots \\ \vdots & \ddots \end{pmatrix}, \quad color \Rightarrow \begin{pmatrix} (231, 103, 92) & \dots \\ \vdots & \ddots \end{pmatrix}$$

# Mat 클래스 개요

- 채널(channel) : 행렬 원소를 구성하는 각각의 값
  - 그레이스케일 영상은 1채널 행렬, 컬러영상은 3채널 행렬로 저장
- 행렬원소의 타입(type) : 행렬원소의 깊이와 채널수를 합쳐 놓은 것
- 행렬원소의 타입(type)은 다음과 같은 형식의 기호상수(매크로 상수)로 표현함

`CV_<bit-depth>{U|S|F}C(<number_of_channels>)`

- 깊이 기호상수 뒤에 C1, C3 같은 채널 정보가 추가되는 형태임
- **CV\_8UC1** : 8비트 unsigned char 자료형을 사용하고 채널이 1개인 행렬을 의미함, 그레이스케일 영상의 type임
- **CV\_8UC3** : unsigned char 자료형을 사용하고 채널이 3개인 행렬 을 의미함, B, G, R 3개의 색성분을 가지고 있는 컬러 영상의 type임
- **CV\_32FC2** : 복소수처럼 두 개의 실수 값을 사용하는 행렬에 사용

# Mat 객체의 생성과 초기화

```
Mat::Mat();
```

- 가장 기본적인 방법은 기본생성자를 이용하는 방법임

```
Mat img1;
```

- 원소가 없는 비어 있는 행렬을 생성할 때 사용
- Mat 객체를 생성하고 멤버변수 rows, cols, data 값을 0으로 초기화
- 행렬의 원소데이터를 저장하기 위한 메모리 공간은 생성하지 않음
- 객체 img1은 원소가 존재하지 않으므로 영상 처리 함수의 입력으로 사용하거나 또는 원소 값을 참조하려고 하면 에러가 발생

img1

```
dims  
rows => 0  
cols => 0  
data => 0  
...
```

# Mat 객체의 생성과 초기화

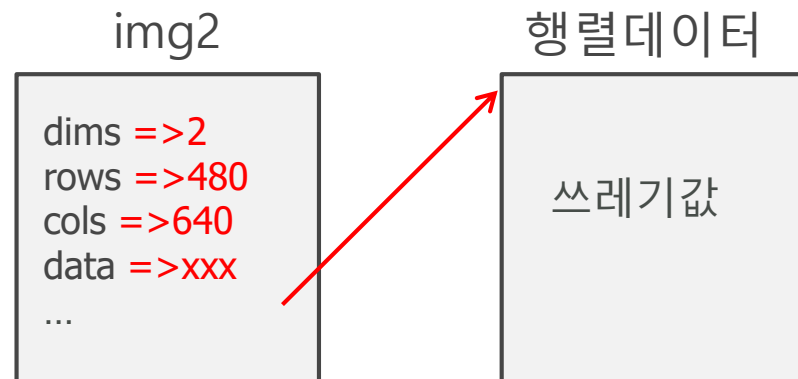
```
Mat::Mat(int rows, int cols, int type);
```

- **rows**      새로 만들 행렬의 행 개수(영상의 세로 크기)
- **cols**      새로 만들 행렬의 열 개수(영상의 가로 크기)
- **type**      새로 만들 행렬의 타입

- 저장할 행렬의 행의 개수, 열의 개수, 타입을 지정하는 방식

```
Mat img2(480, 640, CV_8UC1);      // unsigned char, 1-channel
Mat img3(480, 640, CV_8UC3);      // unsigned char, 3-channels
```

- Mat 객체를 생성하고 멤버 변수를 매개 변수값으로 초기화
- 행렬원소데이터를 저장하기 위한 메모리 공간을 동적할당, 모든 원소는 쓰레기값을 갖는다.



# Mat 객체의 생성과 초기화

- 세 번째 인자 type에는 행렬의 타입(type)을 나타내는 기호상수(매크로 상수)를 전달함
- **CV\_8UC1** : 행렬의 원소가 uchar 자료형이고 1개의 채널을 가짐
- **CV\_8UC3** : 행렬의 원소가 uchar 자료형이고 3개의 채널을 가짐
- **CV\_8UC1** 타입은 그레이스케일 영상을 저장할 때, **CV\_8UC3** 타입은 컬러 영상에서 저장할 때 사용함

# Mat 객체의 생성과 초기화

```
Mat::Mat(Size size, int type);
```

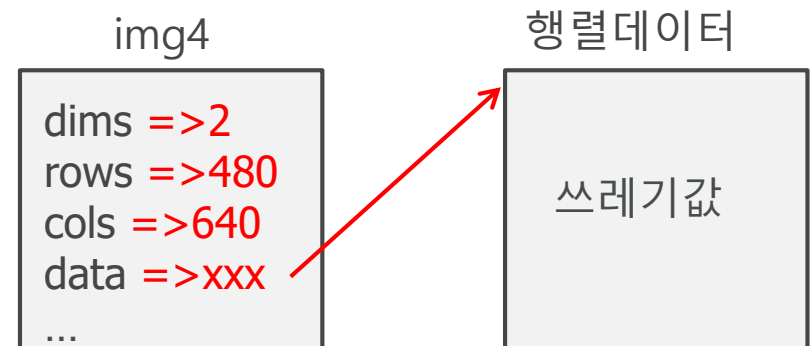
- **size**      새로 만들 행렬의 크기. Size(cols, rows) 또는 Size(width, height)
- **type**      새로 만들 행렬의 타입

- 행렬의 크기를 지정할 때 Size 클래스를 사용할 수도 있음
- Size 클래스의 width는 cols을 의미, height 는 rows를 의미

```
Size sz(640,480);           // 640: 열의 갯수, 480 : 행의 갯수
Mat img4(sz, CV_8UC3);
```

- Size(640,480) : Size 클래스의 생성자를 직접 호출, Size 임시객체를 생성해주고 해당문장 실행 후 사라짐

```
Mat img4(Size(640, 480), CV_8UC3);
```





# Mat 객체의 생성과 초기화

```
Mat::Mat(int rows, int cols, int type, const Scalar& s);
Mat::Mat(Size size, int type, const Scalar& s);
```

- **rows**      새로 만들 행렬의 행 개수(영상의 세로 크기)
- **cols**      새로 만들 행렬의 열 개수(영상의 가로 크기)
- **size**      새로 만들 행렬의 크기
- **type**      새로 만들 행렬의 타입
- **s**          행렬 원소 초깃값

- Mat 객체를 생성함과 동시에 행렬의 모든 원소 값을 특정 값으로 초기화 해줌
- 객체 img5의 모든 원소값을 128으로 초기화, 객체 img6의 모든 원소값을 (0,0,255)로 초기화

```
Mat img5(480, 640, CV_8UC1, Scalar(128));      // initial values, 128
Mat img6(480, 640, CV_8UC3, Scalar(0, 0, 255));      // initial values, red
```

# Mat 객체의 생성과 초기화

- `Scalar(128)` : Scalar 객체 생성 후 해당문장 실행 후 사라짐, 그레이스케일 영상의 픽셀값을 표현할 때에는 Scalar클래스의 첫번째 원소만을 사용함

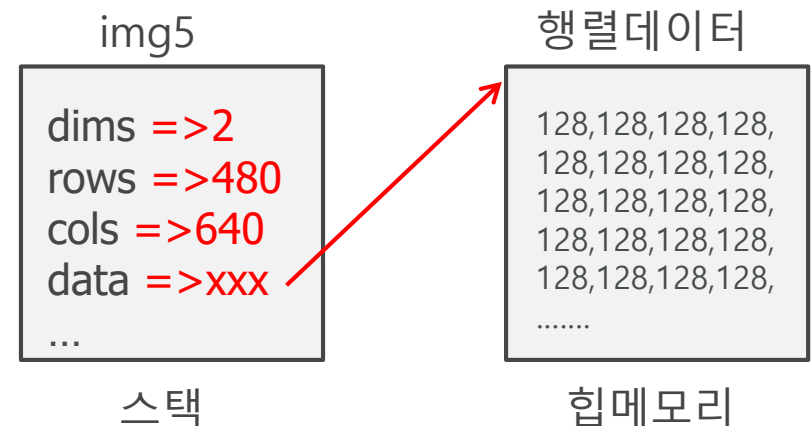
`Scalar gray(128);`

`Mat img5(480,640, CV_8UC1, gray);`

- `Scalar(0,0,255)` : 컬러 영상의 픽셀 값을 표현할 때에는 Scalar클래스의 3개의 원소만 사용, 파란색(B), 녹색(G), 빨간색(R) 순서로 저장함

`Scalar color(0,0,255);`

`Mat img6(480,640, CV_8UC3, color);`



# Mat 객체의 생성과 초기화

```
Mat::Mat(int rows, int cols, int type, void* data, size_t step=AUTO_STEP);
Mat::Mat(Size size, int type, void* data, size_t step=AUTO_STEP);
```

- **rows**      새로 만들 행렬의 행 개수(영상의 세로 크기)
- **cols**      새로 만들 행렬의 열 개수(영상의 가로 크기)
- **size**      새로 만들 행렬의 크기
- **type**      새로 만들 행렬의 타입
- **data**      사용할 (외부) 행렬 데이터의 주소. 외부 데이터를 사용하여 Mat 객체를 생성할 경우, 생성자에서 원소 데이터 저장을 위한 메모리 공간을 동적으로 할당하지 않습니다.
- **step**      (외부) 행렬 데이터에서 한 행이 차지하는 바이트 수. 만약 외부 행렬 데이터의 각 행에 여분의 패딩 바이트(padding byte)가 존재한다면 명시적으로 지정해야 합니다. 만약 기본값 AUTO\_STEP을 사용하면 패딩 바이트가 없다고 간주합니다.

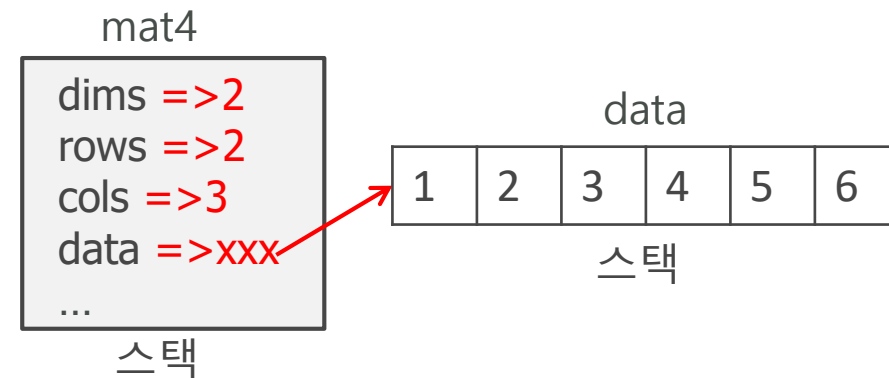
- 객체를 생성할 때, 행렬 원소를 저장할 메모리 공간을 새로 할당하는 것이 아니라 기존에 이미 할당되어 있는 메모리 공간의 데이터를 행렬 원소 값으로 사용할 수 있음

# Mat 객체의 생성과 초기화

- 일반적으로 행렬원소로 사용할 데이터를 배열로 선언하고 배열의 시작 주소를 생성자에 전달

```
float data[] = { 1, 2, 3, 4, 5, 6 };
Mat mat4(2, 3, CV_32FC1, data);
```

$$\text{mat4} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$



- Mat 객체를 생성할 때 자체적인 메모리 할당을 수행하지 않고 외부 메모리를 참조하는 방식이기 때문에 객체 생성이 빠르다는 장점
- 외부 배열 크기와 생성할 행렬 원소 개수는 같아야 하고 자료형도 같아야 함

# Mat 객체의 생성과 초기화

- Mat\_ 클래스를 사용하는 방법
- Mat\_ 클래스는 Mat 클래스를 상속하여 만든 템플릿 클래스로서 Mat\_ 클래스 객체와 Mat 객체는 상호 변환이 가능함
- Mat\_ 클래스는 << 연산자와 콤마(,)를 이용하여 간단하게 행렬 원소 값을 설정가능 함

```

Mat_<float> mat5_(2, 3);
mat5_ << 1, 2, 3, 4, 5, 6;
Mat mat5 = mat5_;

```

→ `Mat mat5 = (Mat_<float>(2, 3) << 1, 2, 3, 4, 5, 6);`

$$\text{mat5} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$
 mat5 행렬은 2×3 크기  
타입은 CV\_32FC1임

# Mat 객체의 생성과 초기화

- 기타방법

- 전용 함수를 이용하는 방법

```
Mat mat1 = Mat::zeros(3, 3, CV_32SC1);    // 0's matrix
Mat mat2 = Mat::ones(3, 3, CV_32FC1);     // 1's matrix
Mat mat3 = Mat::eye(3, 3, CV_32FC1);      // identity matrix
```

- OpenCV 4.0에서는 C++11의 초기화 리스트(initializer list)를 이용한  
행렬 초기화 방법을 사용할 수 있음

```
Mat mat6 = Mat_<float>({2, 3}, { 1, 2, 3, 4, 5, 6 });
```

- Mat 클래스의 Mat::create() 멤버 함수를 사용하여 객체 생성가능

```
mat4.create(256, 256, CV_8UC3); // 256x256, uchar, 3-channels
mat5.create(4, 4, CV_32FC1);    // 4x4, float, 1-channel
```

# 행렬의 전체 원소값 수정

```
Mat& Mat::operator = (const Scalar& s);
```

- **s**            행렬 원소에 설정할 값
- **반환값**    값이 설정된 Mat 객체의 참조

```
Mat& Mat::setTo(InputArray value, InputArray mask = noArray());
```

- **value**        행렬 원소에 설정할 값
- **mask**        마스크 행렬. 마스크 행렬의 원소가 0이 아닌 위치에서만 value 값이 설정됩니다. 행렬 전체 원소 값을 설정하려면 noArray() 또는 Mat()을 지정합니다.
- **반환값**        Mat 객체의 참조

- Mat 클래스는 = 연산자 또는 Mat::setTo() 멤버 함수를 이용하여 행렬 전체 원소 값을 한꺼번에 설정할 수 있음

# 행렬의 전체 원소값 수정

- Mat 객체의 원소값을 수정할 때 Scalar객체를 이용하여 값을 표현함

```
Mat mat1(100, 100, CV_8UC1, Scalar(0));  
mat1 = Scalar(255);           //모든 원소에 255대입  
mat1.setTo(Scalar(255));
```

```
Mat mat2(100, 100, CV_8UC3, Scalar(0, 0, 0));  
mat2 = Scalar(255, 255, 255); //모든 원소에 (255,255,255) 대입  
mat2.setTo(Scalar(255,255,255));
```



# 행렬원소 출력하기

```
static inline  
std::ostream& operator << (std::ostream& out, const Mat& mtx)
```

- `out` C++ 표준 출력 스트림 객체
- `mtx` 출력할 행렬
- 반환값 C++ 표준 출력 스트림 객체의 참조

- OpenCV는 << 연산자 재정의를 이용하여 행렬 원소를 출력하는 기능을 제공함
- << 연산자 왼쪽에는 `std::cout`을 적고, << 연산자 오른쪽에는 `Mat` 객체 변수 이름을 적으면 해당 행렬 원소가 모두 콘솔 창에 출력됨

# 행렬원소 출력하기

- 예를 들어 작은 크기의 행렬 원소를 모두 화면에 출력하는 코드

```
float data[] = { 2.f, 1.414f, 3.f, 1.732f };  
Mat mat1(2, 2, CV_32FC1, data);  
  
std::cout << mat1 << std::endl;
```

- 대괄호 안에 행렬 형태로 출력됨, 행을 세미콜론으로 구분

```
[2, 1.414;  
 3, 1.732]
```

# 행렬원소 출력하기

```
void imshow(const String& winname, InputArray mat);
```

- **winname**      영상을 출력할 대상 창 이름
- **mat**          출력할 영상 데이터(Mat 객체)

- 1채널 또는 3채널 행렬을 영상으로 출력할 때는 imshow함수 사용
- 행렬원소의 type이 **CV\_8UC1**인 경우 원소값을 밝기값으로 해석하여 출력해줌, 0은 검정색, 255는 흰색으로 출력
- 행렬원소의 type이 **CV\_8UC3**인 경우 3개의 원소값을 파란색(Blue), 녹색(Green), 빨간색(Red) 성분으로 해석하여 해당 색상을 출력
- 행렬원소가 부호 없는 16비트 또는 32비트 정수형이라면 행렬 원소 값을 256으로 나눈 값을 영상의 밝기값으로 해석하여 출력
- 행렬원소가 32비트 또는 64비트 실수형 행렬이라면 행렬 원소에 255를 곱한 값을 밝기값으로 해석하여 출력

## 코드3-7 : Mat 객체 생성 및 초기화

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat mat1;                                // empty matrix
    Mat mat2(2, 3, CV_8UC1);                 // uchar, 1ch, 쓰레기값
    Mat mat3(2, 3, CV_8UC1, Scalar(128));    // 1ch, 128로 초기화
    //Mat mat3(Size(3,2), CV_8UC1, Scalar(128)); // 1ch, 128로 초기화
    Mat mat4(2, 3, CV_8UC3, Scalar(0, 0, 255)); // 3ch, (0,0,255)로 초기화
    //Mat mat4(Size(3, 2), CV_8UC1, Scalar(128)); // 1ch, 128로 초기화
    float data[] = { 1, 2, 3, 4, 5, 6 };
    Mat mat5(2, 3, CV_32FC1, data);
    // cout << mat1 << endl;                // error
    cout << mat2 << endl;
    cout << mat3 << endl;
    cout << mat4 << endl;
    cout << mat5 << endl;
}
```

## 코드3-7 : Mat 객체 생성 및 초기화

```
Mat img1(480, 640, CV_8UC1);           // unsigned char, 1-channel
Mat img2(480, 640, CV_8UC3);           // unsigned char, 3-channels
Mat img3(Size(640, 480), CV_8UC3);     // Size(width, height)
Mat img4(480, 640, CV_8UC1, Scalar(128)); // initial values, 128
Mat img5(480, 640, CV_8UC3, Scalar(255, 0, 0)); // initial values, blue
Mat img6(480, 640, CV_8UC3, Scalar(0, 255, 0)); // initial values, Green
Mat img7(480, 640, CV_8UC3, Scalar(0, 0, 255)); // initial values, red

imshow("img1", img1);
imshow("img2", img2);
imshow("img3", img3);
imshow("img4", img4);
imshow("img5", img5);
imshow("img6", img6);
imshow("img7", img7);

waitKey();           // imshow함수 뒤에 반드시 호출해야 함
return 0;

}
```

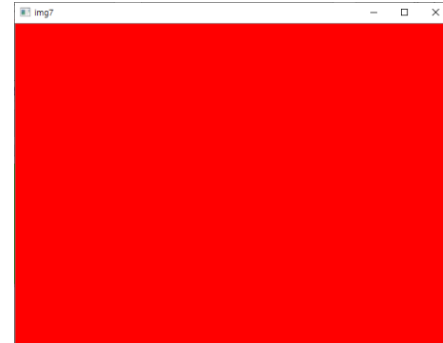
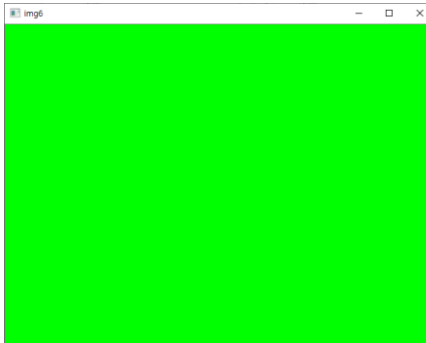
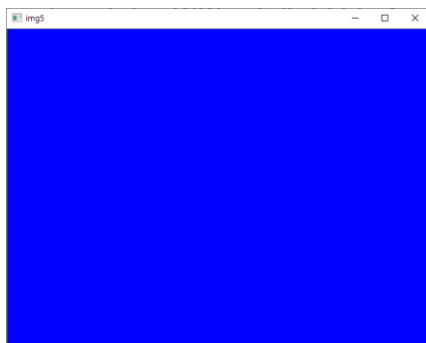
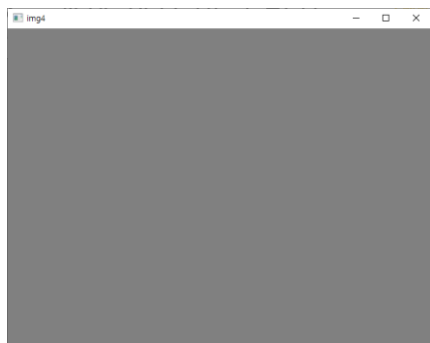
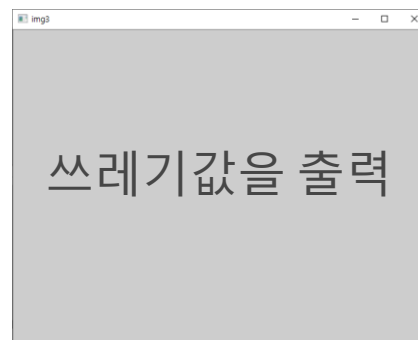
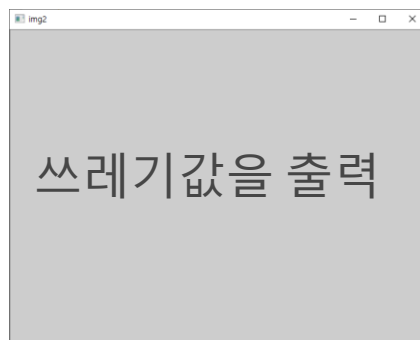
## 코드3-7 : Mat 객체 생성 및 초기화

```

D:\Users\2sungryul\Dropbox\Lecture\2020년2학기\프로그래밍언어응용\강의노트\OpenCV\64\...
[205, 205, 205;
 205, 205, 205]
[128, 128, 128;
 128, 128, 128]
[ 0, 0, 255, 0, 0, 255, 0, 0, 255;
 0, 0, 255, 0, 0, 255, 0, 0, 255]
[1, 2, 3;
 4, 5, 6]

```

# 코드3-7 : Mat 객체 생성 및 초기화

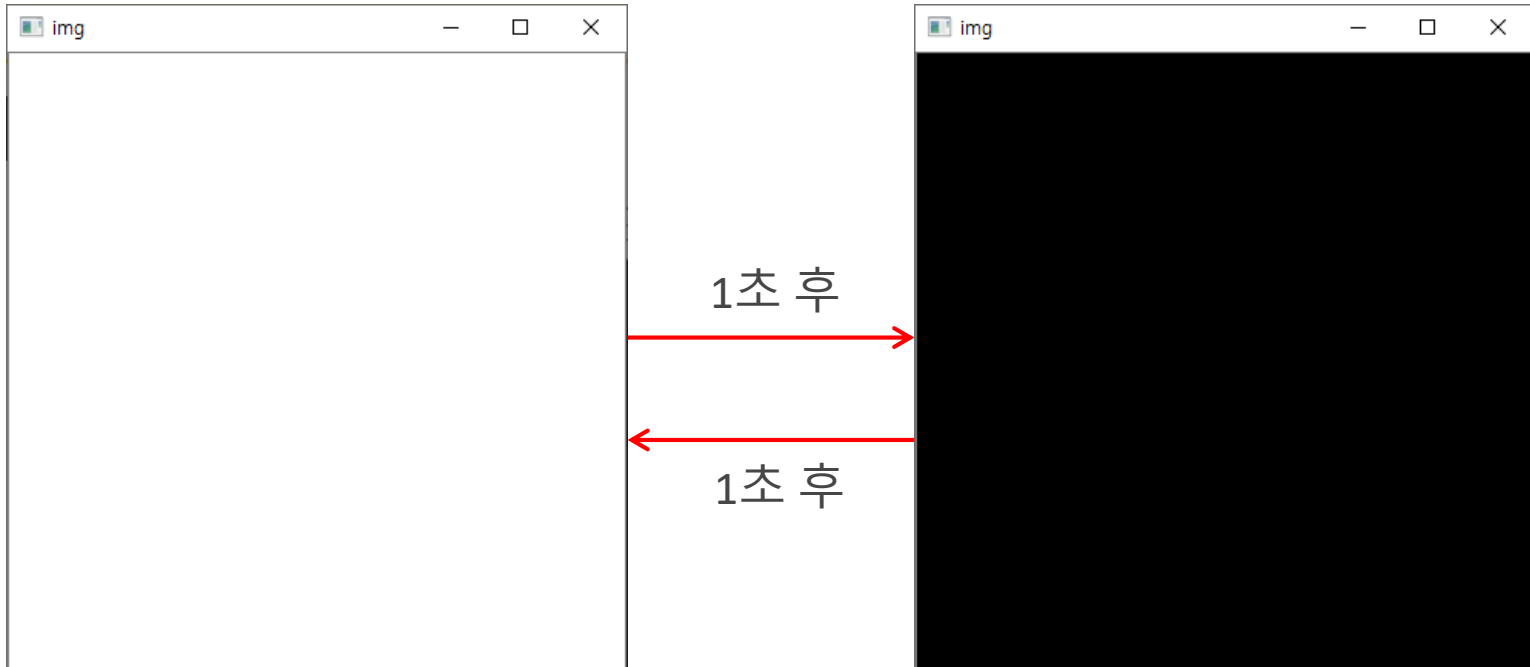


## 예제: 1초마다 화면배경 바꾸기

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat img(400, 400, CV_8UC1);
    int pixel[2] = { 0, 255 };
    int i = 0, key;
    while (true)
    {
        img = Scalar(pixel[i++ % 2]); //pixel[0]와 pixel[1]을 반복한다.
        imshow("img", img);
        key = waitKey(1000);
        if (key == 'q') break;        //q 키누르면 종료
    }
    return 0;
}
```



# 예제: 1초마다 화면배경 바꾸기



# 과제1

- 코드3-7을 참고하여 Mat 객체를 생성하여 다음 행렬들을 저장하고 화면에 출력하는 코드를 작성하라.

$$mat1 = \begin{pmatrix} 3.5 & 2.1 \\ -1.5 & -6.5 \end{pmatrix}$$

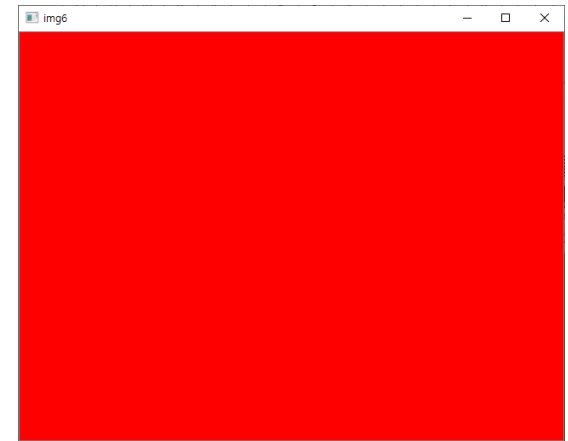
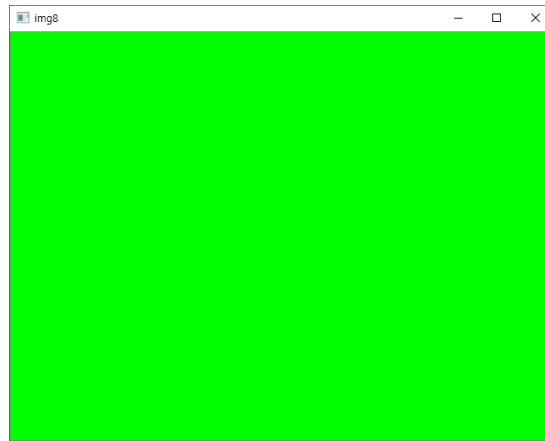
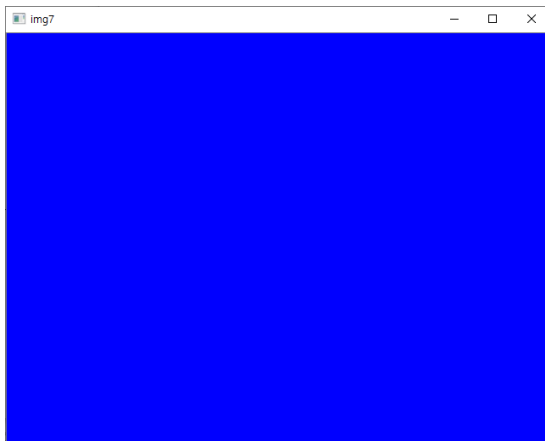
$$mat2 = \begin{pmatrix} 0 & 2 & -1 \\ 5 & 10 & 8 \\ 6 & -7 & 9 \end{pmatrix}$$

$$mat3 = (1 \quad 2 \quad 3 \quad 4)$$

$$mat4 = \begin{pmatrix} 5 \\ 6 \\ 7 \\ 8 \end{pmatrix}$$

## 과제2

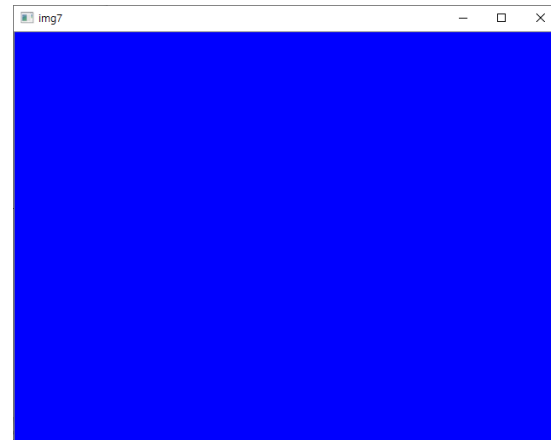
- 코드3-7을 참고하여 다음 결과가 나오도록 코드를 작성하라. 윈도우 크기는 400X300이고 색상은 Blue, Green, Red이다.
- 힌트: Mat 객체 3개를 생성하고 초기값을 각각 `Scalar(255,0,0)`, `Scalar(0,255,0)`, `Scalar(0,0,255)`으로 설정한 후 `imshow`함수 사용



## 과제3

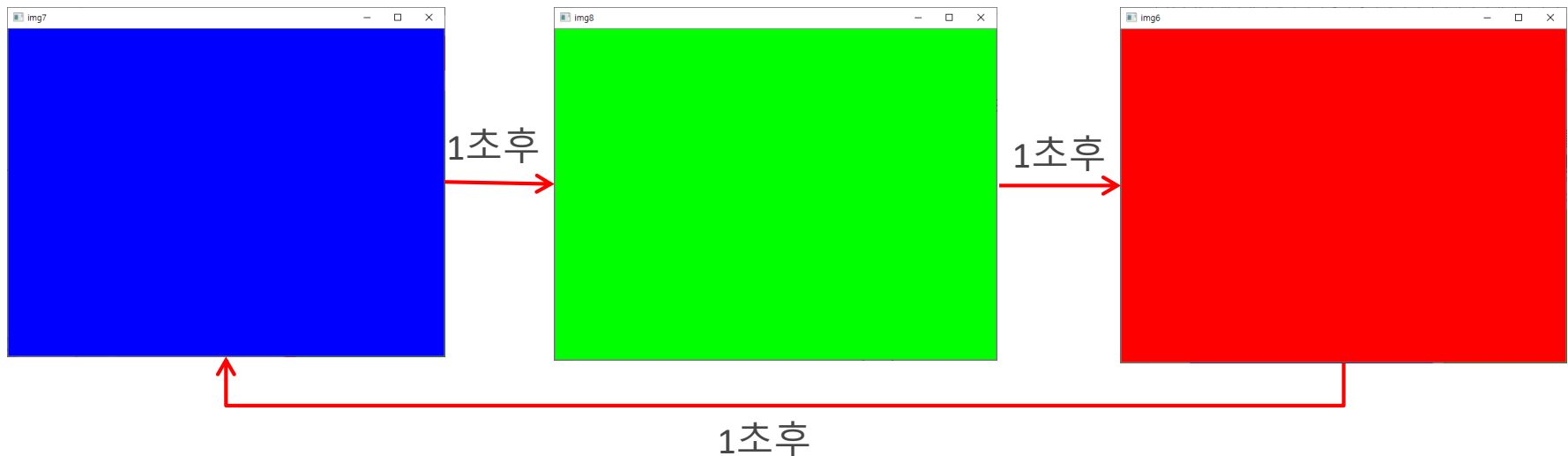
- 키보드로부터 컬러값을 입력받아 화면의 배경색을 변경해주는 코드를 작성하라. 윈도우 크기는 400X300이고 초기색은 흰색으로 설정하라.
- 힌트: =연산자 또는 setTo 멤버함수를 이용하라.

B값을 입력하라: 255<엔터>  
G값을 입력하라: 0<엔터>  
R값을 입력하라: 0<엔터>



## 과제4

- 1초마다 화면 배경 바꾸기 예제를 수정하여 다음처럼 윈도우의 배경이 1초 주기로 Blue->Green->Red->Blue->Green->...를 무한 반복하고 q 키를 누르면 종료하는 코드를 작성하시오. 실행결과를 동영상으로 제출할 것
- 반복문은 1번만 사용하고 같은 함수를 2번 이상 호출하지 않도록 해볼 것
- 힌트: Mat 객체를 3채널 행렬로 생성하고 행렬의 원소값을 1초 주기로 `Scalar(255,0,0)`, `Scalar(0,255,0)`, `Scalar(0,0,255)` 순서로 변경해준다.



## 과제5(선택문제)

- 초기에 400x400 크기의 검정색 배경의 윈도우를 생성하고 5msec마다 화면 배경색의 그레이 레벨을 1씩 증가시킨다. 배경색이 흰색이 되면 다시 5msec마다 1씩 감소시킨다. 그리고 다시 검정색이 되면 다시 1씩 증가시킨다. 이과정을 무한히 반복한다. Q를 누르면 종료하도록 한다.
- 그레이 레벨 변환 : 0->1->2->...->255->254->253->...->1->0->1->...
- 반복문은 1번만 사용하고 같은 함수를 2번 이상 호출하지 않도록 해볼 것
- 실행결과를 동영상으로 제출하라.

# 과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.